FAST NUCES — KARACHI CAMPUS

Department of Computer Science

# Bank & ATM Simulation System

## Concurrent IPC-Based Banking Application

### Project Report

Operating Systems — CS-2006



| Roll Number | Name |
|---|---|
| 23K-2027 | Hanzala Ramzan |
| 24K-0792 | Ali Roman |
| 24K-0535 | Ghulam Mujtaba |



Spring 2026

# 1. Project Overview

This project implements a concurrent Bank and ATM simulation using POSIX Inter-Process Communication (IPC) primitives in C. The system models a realistic banking environment where multiple ATM clients can simultaneously communicate with a central bank server to perform financial transactions safely and reliably.

The system is built around three core OS concepts:

- Message Queues (POSIX mqueue) — for ATM-to-Bank communication
- Semaphores — for mutual exclusion on individual account records
- Processes — each ATM runs as a separate process

## 1.1 System Architecture

The architecture follows a Client-Server model:

| Component | Role | IPC Mechanism |
|---|---|---|
| bank_server | Central bank — manages 5 accounts, processes all requests | POSIX Message Queue (reader) |
| atm (client) | ATM terminal — sends user requests to the bank | POSIX Message Queue (writer) |
| runs_atms.sh | Launches multiple ATM processes in separate terminals | Shell / fork |

## 1.2 Supported Operations

| Operation | Description |
|---|---|
| Deposit | Credit an amount to a specified account after PIN verification |
| Withdraw | Debit an amount from an account if balance is sufficient |
| View Account | Display account number, balance, DOB, and last access time |
| Transfer | Move funds between two accounts atomically |
| Change PIN | Update account PIN after old PIN verification |
| Change DOB | Update date of birth field on the account |

# 2. Design & Key Concepts

## 2.1 POSIX Message Queue (IPC)

The message queue named /bank_queue acts as the central communication channel. The bank server opens it in read mode (O_RDONLY | O_CREAT) at startup, while each ATM opens it in write-only mode (O_WRONLY). Messages are plain ASCII strings encoding the operation, account index, amount, and PIN.

Message format examples:

```
    struct mq_attr attr = {
        .mq_flags = 0,
        .mq_maxmsg = 10,
        .mq_msgsize = MAX_MSG_SIZE,
        .mq_curmsgs = 0};

    // Initialize message queue
    mq_unlink(QUEUE_NAME);
    mq = mq_open(QUEUE_NAME, O_CREAT | O_RDONLY, 0644, &attr);
    if (mq == -1)
    {
        perror("[ERROR] mq_open failed");
        exit(EXIT_FAILURE);
    }
```

## 2.2 Semaphores for Account-Level Mutual Exclusion

Each of the 5 accounts has its own POSIX unnamed semaphore (sem_t account_sems[5]). Before any read or write on an account, the server acquires that account's semaphore and releases it after the operation. This allows concurrent operations on different accounts while preventing race conditions on the same account.

```
void process_deposit(int account_index, int amount, const char *pin)
{
    bool success = false;

    sem_wait(&account_sems[account_index]);


    sem_post(&account_sems[account_index]);
    log_transaction("DEPOSIT", account_index, amount, success);
}
```

## 2.3 Deadlock Prevention in Transfers

The transfer operation must lock two accounts simultaneously. To prevent deadlock (thread A holds lock 1, waits for lock 2; thread B holds lock 2, waits for lock 1), the server always acquires locks in ascending index order regardless of which is source or destination:

```
    }

    // Lock accounts in order to prevent deadlock
    int first = from_account < to_account ? from_account : to_account;
    int second = from_account < to_account ? to_account : from_account;

    sem_wait(&account_sems[first]);
    sem_wait(&account_sems[second]);
```

```
    sem_post(&account_sems[second]);
    sem_post(&account_sems[first]);
    log_transaction("TRANSFER", from_account, amount, success);
}
```

## 2.4 Input Validation (Client-Side)

The ATM client performs thorough validation before sending any request to the server:

- PIN: exactly 4 decimal digits (validate_pin function)
- DOB: strict DD/MM/YYYY format with range checks on day, month, year (1900-2100)
- Amount: must be a positive integer
- Account index: 0 to 4 inclusive
- All scanf calls check return values; invalid input triggers clear_input_buffer() and re-prompts

# 3. Important Code Sections

## 3.1 Bank Server — Account Initialization

Accounts are randomly initialized with unique numbers, balances, PINs, and DOBs. Each gets its own semaphore:

```
void initialize_accounts()
{
    srand(time(NULL));

    for (int i = 0; i < ACCOUNT_COUNT; i++)
    {
        accounts[i].account_num = 10000 + (rand() % 90000);
        accounts[i].balance = 1000 + (rand() % 9000);
        snprintf(accounts[i].pin, PIN_LENGTH + 1, "%04d", rand() % 10000);
        snprintf(accounts[i].dob, DOB_LENGTH, "%02d/%02d/%04d",
                 1 + rand() % 28, 1 + rand() % 12, 1950 + rand() % 50);
        accounts[i].is_active = true;
        accounts[i].last_access = time(NULL);

        sem_init(&account_sems[i], 0, 1);
```

## 3.2 Message Queue Setup (Server)

```
    struct mq_attr attr = {
        .mq_flags = 0,
        .mq_maxmsg = 10,
        .mq_msgsize = MAX_MSG_SIZE,
        .mq_curmsgs = 0};

    // Initialize message queue
    mq_unlink(QUEUE_NAME);
    mq = mq_open(QUEUE_NAME, O_CREAT | O_RDONLY, 0644, &attr);
```

## 3.3 ATM — Sending a Request

```c
void send_request(mqd_t mq, const char *request, const char *req)
{
    printf("[*] Sending: %s\n", req);
    if (mq_send(mq, request, strlen(request) + 1, 0) == -1)
    {
        perror("[ERROR] mq_send failed");
    }
    else
    {
        printf("[*] Request sent successfully\n");
    }

}
```

## 3.4 Server — Message Dispatcher

The handle_message function parses the action field and routes to the correct handler:

```c
    if (strcmp(action, "deposit") == 0)
    {
        if (sscanf(message, "%d deposit %d %4s", &account_index, &amount, pin) == 3)
        {
            process_deposit(account_index, amount, pin);
        }
    }
    else if (strcmp(action, "withdraw") == 0)
    {
        if (sscanf(message, "%d withdraw %d %4s", &account_index, &amount, pin) == 3)
        {
            process_withdrawal(account_index, amount, pin);
        }
```

## 3.5 Graceful Shutdown (SIGINT Handler)

```c
void sigint_handler(int sig)
{
    if (sig == SIGINT)
    {
        printf("\n[*] Received SIGINT, shutting down gracefully...\n");
        shutdown_flag = 1;
        exit(0);
    }
}

void cleanup_resources()
{
    printf("[*] Cleaning up resources...\n");

    // Close and unlink message queue
    if (mq != -1)
    {
        mq_close(mq);
        mq_unlink(QUEUE_NAME);
    }
```

# 4. Data Structures

## 4.1 Account Structure

```
typedef struct
{
    int account_num;
    int balance;
    char pin[PIN_LENGTH + 1];
    char dob[DOB_LENGTH];
    bool is_active;
    time_t last_access;
} Account;
```

## 4.2 Key Constants

| Constant | Value | Purpose |
|---|---|---|
| QUEUE_NAME | /bank_queue | POSIX MQ identifier |
| MAX_MSG_SIZE | 256 | Max bytes per message |
| ACCOUNT_COUNT | 5 | Number of accounts |
| PIN_LENGTH | 4 | Digits in PIN |
| DOB_LENGTH | 11 | DD/MM/YYYY + null terminator |

# 5. Compilation & Execution

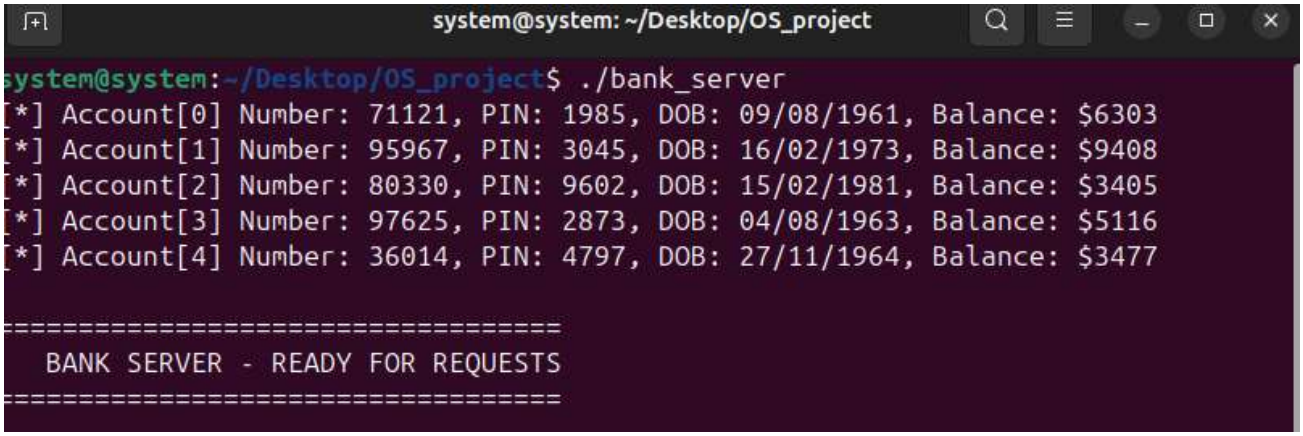
## 5.1 Build Commands

```
 # Compile the bank server

              gcc bank_server.c -o bank_server -lpthread -lrt

# Compile the ATM client

              gcc atm.c -o atm -lrt
```

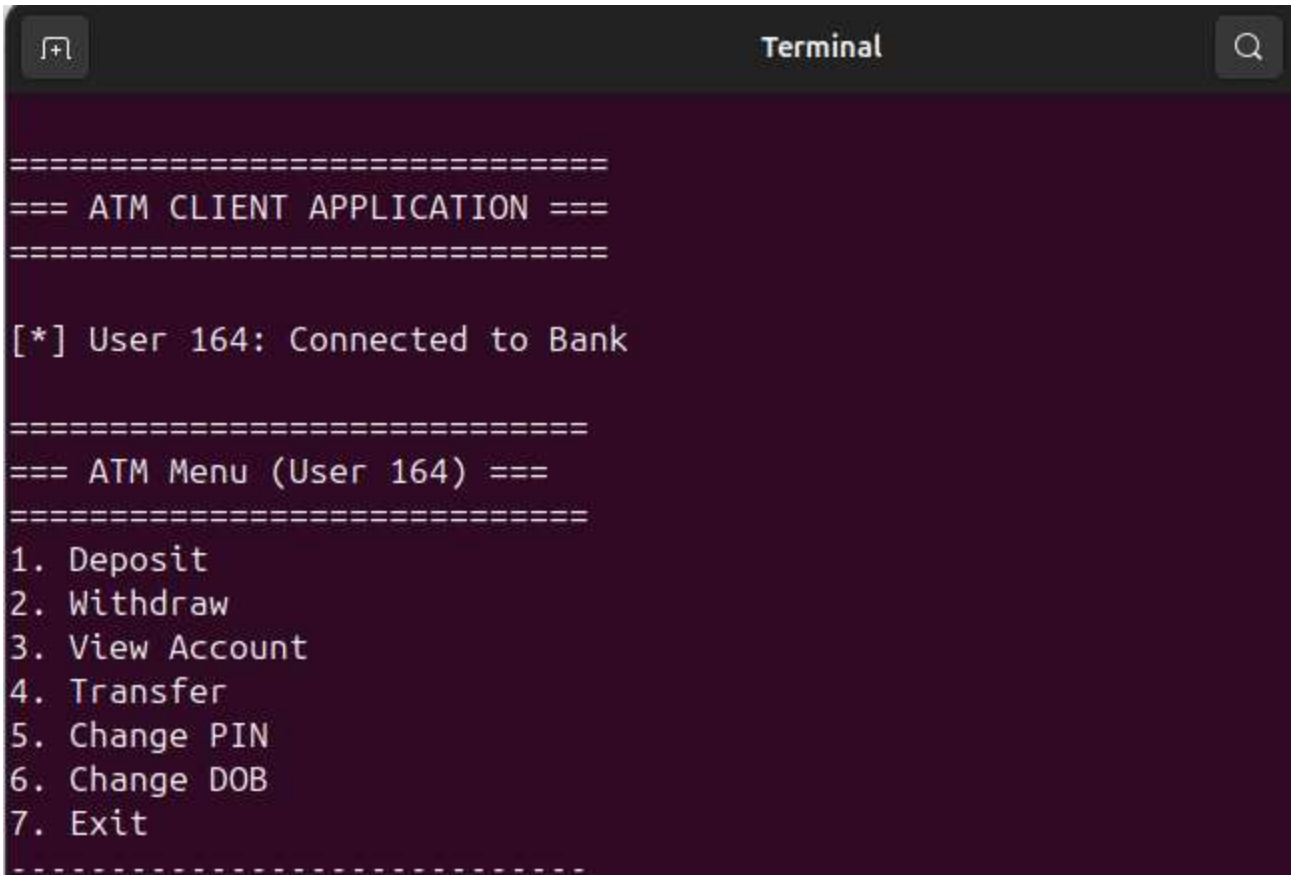
## 5.2 Running the System

```
# Step 1: Start the bank server (in Terminal 1)
             ./bank_server

# Step 2: Start ATM client(s) (in separate terminals)
             ./atm

# OR: Launch multiple ATM clients at once using the script
             chmod +x runs_atms.sh && ./runs_atms.sh
```

## 5.3 Expected Server Output on Startup



## 5.4 Expected ATM Output



# 6. Concurrency & Safety Analysis

## 6.1 Race Condition Prevention

Without semaphores, two ATM processes could simultaneously read the same account balance (e.g., $500), both decide a $400 withdrawal is valid, and both subtract, leaving -$300. The per-account semaphores serialize such operations: only one process can hold the semaphore for a given account at a time, making balance checks and updates atomic.

## 6.2 Deadlock Freedom

The global lock ordering (always lock lower-index account first) guarantees deadlock freedom. Any two concurrent transfers will agree on which account to lock first, so neither can be blocked waiting for a lock the other holds.

## 6.3 Signal Safety

The SIGINT handler only sets a volatile sig_atomic_t flag (shutdown_flag). The main loop checks this flag between mq_receive calls and exits cleanly, ensuring mq_unlink and sem_destroy always execute even on Ctrl+C.

## 6.4 Potential Improvements

- Add a response queue so the ATM receives confirmation/error messages from the server
- Persist account state to a file so data survives server restarts
- Add a failed-attempt lockout after N wrong PINs
- Encrypt PIN storage (currently stored as plaintext in memory)
- Replace busy-wait sleep(1) in ATM with blocking response queue reads

# 7. Conclusion

This project demonstrates the practical application of POSIX IPC mechanisms in building a multi-user concurrent system. By combining message queues for inter-process communication with per-account semaphores for mutual exclusion, the system correctly handles simultaneous ATM sessions without data corruption or deadlocks.

The implementation covers all standard banking operations with proper input validation, graceful shutdown, and structured logging. The modular design cleanly separates the ATM client (user interaction + message sending) from the bank server (account management + concurrency control), making the codebase straightforward to extend or maintain.

Key OS concepts demonstrated in this project:

- POSIX Message Queues — asynchronous IPC between unrelated processes
- POSIX Unnamed Semaphores — fine-grained mutual exclusion at the account level
- Signal Handling — safe asynchronous shutdown with resource cleanup
- Process-based concurrency — multiple ATM processes coexisting on a single machine